

Proyecto 2 - Base de Datos.

Introducción:

El presente proyecto documenta el diseño e implementación de un Sistema de Gestión de Inventario y Ventas para una tienda minorista. El sistema permite controlar los productos disponibles en inventario, registrar las ventas realizadas a clientes por parte de los empleados, gestionar las compras efectuadas a los proveedores para reabastecer el stock, y generar reportes operativos que apoyen la toma de decisiones.

La aplicación se desarrolla como una solución web completa que integra una interfaz de usuario, un backend con lógica de negocio y una base de datos relacional implementada en PostgreSQL. Toda la infraestructura se despliega mediante Docker, lo que garantiza un entorno reproducible e independiente del sistema operativo del evaluador. Las credenciales de acceso a la base de datos y otras variables sensibles se gestionan mediante archivos de entorno (.env), siguiendo buenas prácticas de seguridad.

Este documento se enfoca en la fase de diseño de la base de datos, abarcando el análisis del dominio, el diagrama entidad-relación, el modelo relacional y la justificación formal de la normalización hasta la Tercera Forma Normal (3FN).

Análisis:

Categoría: agrupa los productos por tipo (electrónica, abarrotes, ropa, etc.) para facilitar su organización y búsqueda.

Producto: representa cada artículo vendible por la tienda. Incluye información de precio de venta, stock actual y stock mínimo para alertas de reabastecimiento.

Proveedor: entidades externas a las que la tienda compra mercadería para reabastecer su inventario.

Cliente: personas o empresas que realizan compras en la tienda. Se registran para reportes, facturación y fidelización.

Empleado: personal que atiende ventas y registra compras a proveedores.

Usuario: cuenta de acceso al sistema, asociada opcionalmente a un empleado. Permite autenticación mediante username y password hasheado.

Venta: cabecera de una transacción de salida (tienda -> cliente). Registra la fecha, el cliente, el empleado que atendió, el método de pago y el total.

Detalle_Venta: líneas que componen cada venta. Es una entidad débil porque su existencia depende de una venta específica; no tiene sentido un detalle sin la venta a la que pertenece.

Compra: cabecera de una transacción de entrada (proveedor -> tienda). Registra la fecha, el proveedor, el empleado responsable y el total.

Detalle_Compra: líneas que componen cada compra. También es una entidad débil por la misma razón.

Entidades y Atributos:

Entidad: CATEGORIA

- id_categoria (PK)
- nombre
- descripcion

Entidad: PROVEEDOR

- id_proveedor (PK)
- nombre
- nit (UNIQUE)
- telefono
- email
- direccion

Entidad: PRODUCTO

- id_producto (PK)
- codigo (UNIQUE)
- nombre
- descripcion
- precio_venta
- stock
- stock_minimo
- activo
- id_categoria (FK)

Entidad: CLIENTE

- id_cliente (PK)
- nombre
- apellido
- nit
- telefono
- email

Entidad: EMPLEADO

- id_empleado (PK)
- nombre

- apellido
- dpi (UNIQUE)
- telefono
- email
- puesto
- fecha_contratacion
- salario
- activo

Entidad: USUARIO

- id_usuario (PK)
- username (UNIQUE)
- password_hash
- rol
- id_empleado (FK, UNIQUE)
- activo

Entidad: VENTA

- id_venta (PK)
- fecha
- total
- metodo_pago
- estado
- id_cliente (FK)
- id_empleado (FK)

Entidad: DETALLE_VENTA

- id_detalle_venta (PK)
- id_venta (FK)
- id_producto (FK)
- cantidad
- precio_unitario
- subtotal

Entidad: COMPRA

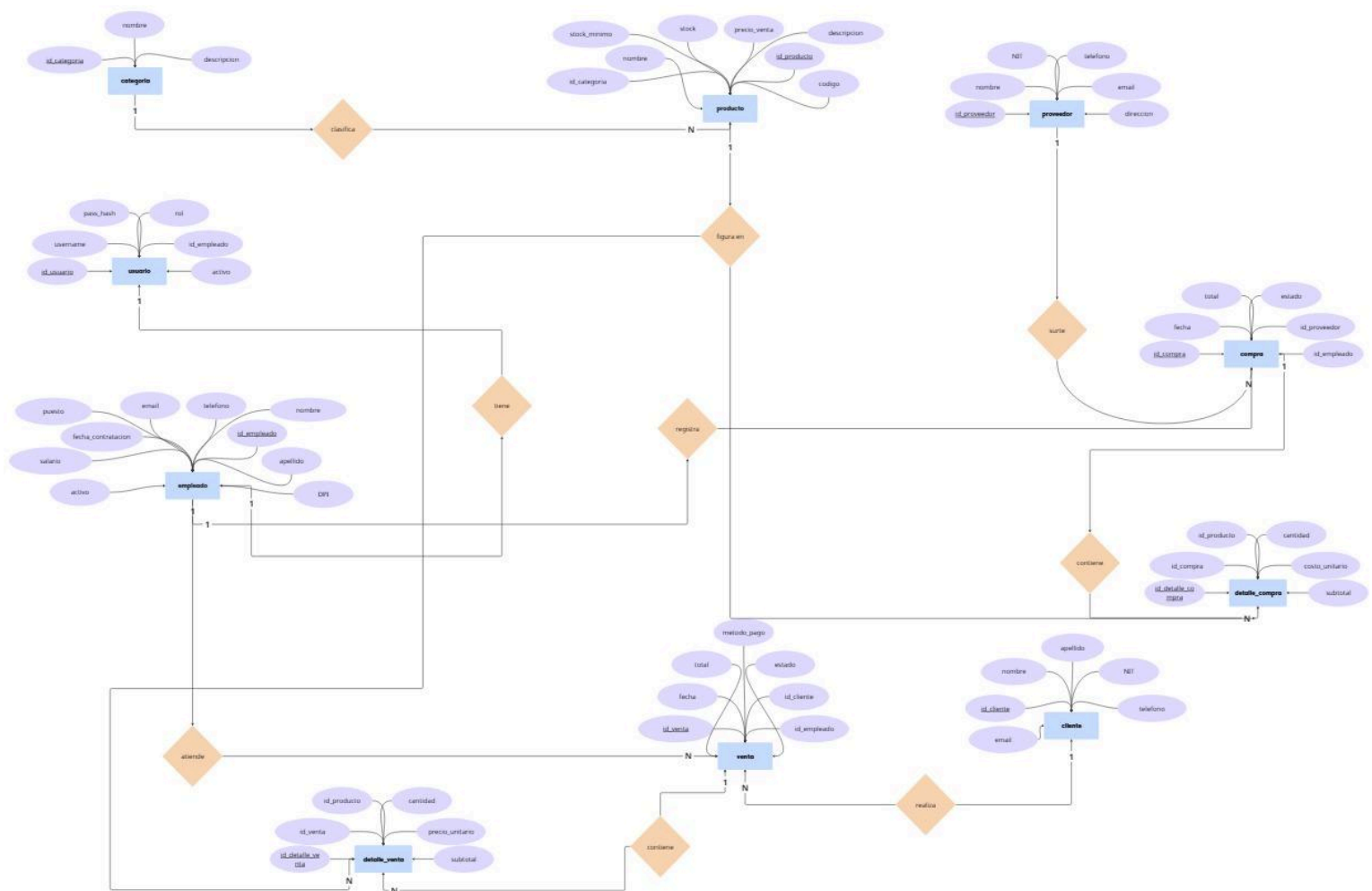
- id_compra (PK)
- fecha
- total
- estado
- id_proveedor (FK)
- id_empleado (FK)

Entidad: DETALLE_COMPRA

- id_detalle_compra (PK)
- id_compra (FK)
- id_producto (FK)
- cantidad
- costo_unitario
- subtotal

DER:

Link: https://miro.com/welcomeonboard/TWtFL0RzTGhGcjd2WCtGYmpJblcyNENaamwrRnluaDNBWmNYRDk1cFpZcStyS2t4Yy9sNFQ4M0lDWURnVFJhckRzUGtHNkNmZUZSaXNKc05VMXVFQ1VVS2h0VTZ0NzB0dzJoekhhT2l0NmtLVFhYMmczeGhSY1lrNzIKSXpNeldQdGo1ZEV3bUdPQWRZUHQzSGl6V2NBPT0hdjE=?share_link_id=15675485392



ER:<https://mermaid.ai/d/4eed2efb-42d3-41cc-b5e5-0ebfa881fa02>

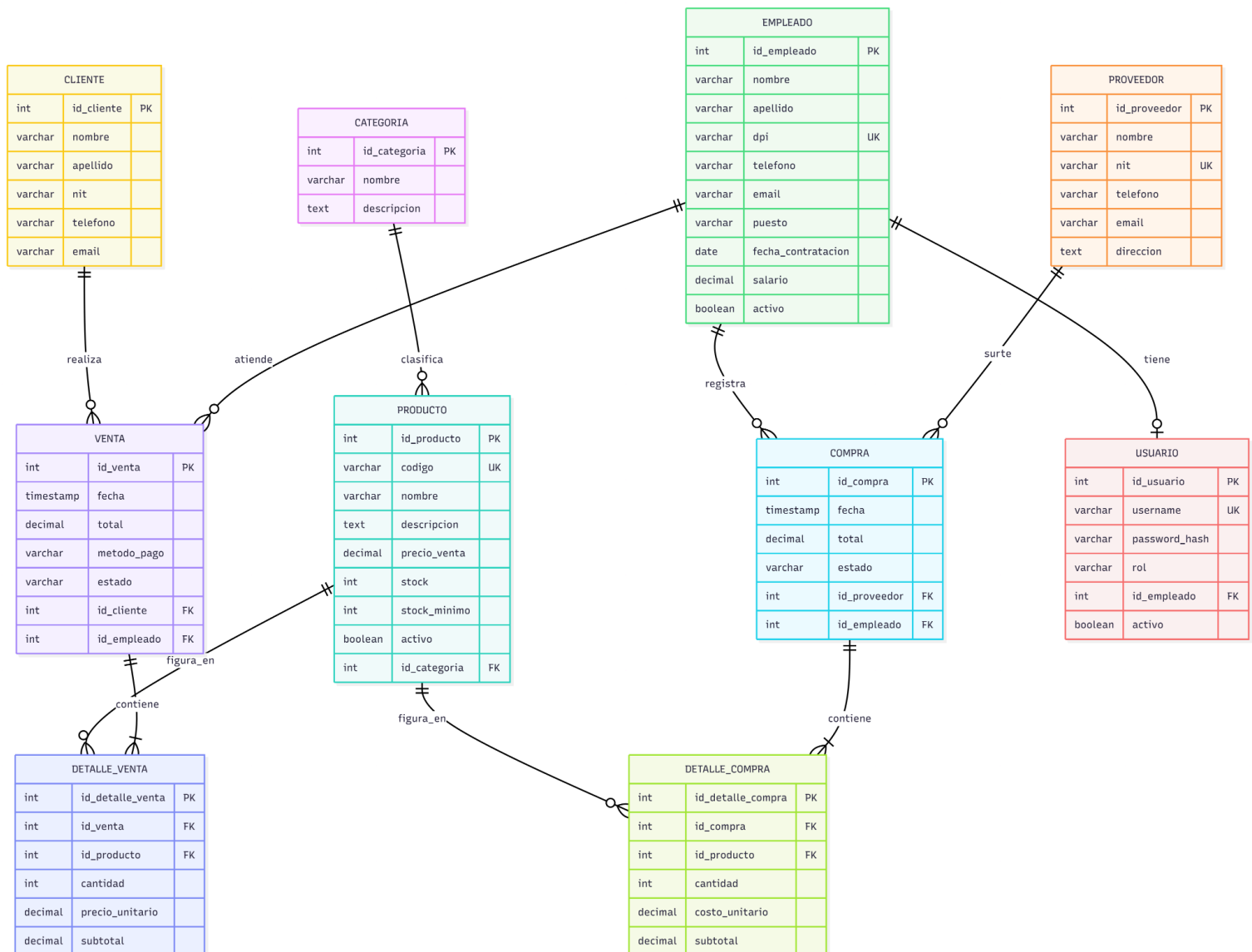


Tabla No Normalizada

Partimos de una única tabla que mezcla toda la información del sistema de ventas e inventario:

OPERACION_COMPLETA(

id_venta,
 fecha_venta,
 metodo_pago,
 cliente_nombre,
 cliente_apellido,
 cliente_nit,
 cliente_telefono,
 cliente_email,
 empleado_nombre,

empleado_apellido,
empleado_dpi,
empleado_puesto,
empleado_salario,
productos_vendidos,
cantidades,
precios_unitarios,
categoria_nombre,
categoria_descripcion,
proveedor_nombre,
proveedor_nit,
proveedor_telefono,
id_compra,
fecha_compra,
costos_unitarios,
username,
password_hash,
rol)

Problemas:

Datos repetidos (el nombre del cliente y del empleado se repite en cada venta)

Listas dentro de un solo campo (varios productos, cantidades y precios en una misma fila)

Difícil de actualizar (cambiar el teléfono de un cliente obliga a modificar múltiples filas)

Inconsistencias (riesgo de escribir distinto el mismo nombre de categoría o proveedor)

Información de ventas y compras mezclada en la misma estructura

Atributos con muchos valores nulos (una fila de venta no usa los campos de compra y viceversa)

Primera Forma Normal (1FN)

Se eliminaron los grupos repetitivos separando los productos de cada venta y cada compra en tablas de detalle independientes, garantizando la atomicidad de todos los atributos.

Proceso:

Cada campo debe tener un solo valor (atomicidad)

Se identifican listas dentro de los campos: productos_vendidos, cantidades, precios_unitarios en las ventas; productos_comprados, cantidades, costos_unitarios en las compras

Solución:

Separar las líneas de productos de cada venta en una tabla independiente

Aquí se creó Detalle_Venta

Separar las líneas de productos de cada compra en otra tabla independiente

Aquí se creó Detalle_Compra

Separar también las operaciones de Venta (cliente → tienda) y Compra (proveedor → tienda) como entidades propias, ya que son transacciones de naturaleza opuesta

Se cumple 1FN**Segunda Forma Normal (2FN)**

Se eliminaron las dependencias parciales separando entidades como Cliente, Empleado y Producto en tablas independientes, y verificando que en las tablas de detalle los atributos no-clave dependan de la clave completa.

Proceso:

No debe haber dependencias parciales de la PK

Las tablas con PK simple (Categoria, Proveedor, Producto, Cliente, Empleado, Usuario, Venta, Compra) cumplen 2FN de forma trivial, ya que no existen claves compuestas que permitan una dependencia parcial

En Detalle_Venta existe una clave candidata compuesta (id_venta, id_producto):

cantidad depende de ambos (el mismo producto puede venderse en cantidades distintas en diferentes ventas)

precio_unitario depende de ambos (el precio puede cambiar entre ventas)

subtotal depende de ambos

En Detalle_Compra ocurre lo mismo con (id_compra, id_producto):

cantidad, costo_unitario y subtotal dependen de la clave completa

Solución:

Los datos del cliente se movieron a la tabla Cliente (no se repiten en cada venta)

Los datos del empleado se movieron a la tabla Empleado (no se repiten en cada transacción)

Los datos del producto se movieron a la tabla Producto (no se repiten en cada línea de detalle)

Se cumple 2FN

Tercera Forma Normal (3FN)

Se eliminaron las dependencias transitivas creando tablas independientes para Categoría, Proveedor y Usuario, evitando redundancia de información y garantizando que los atributos no-clave dependan únicamente de la PK.

Proceso:

No debe haber dependencias transitivas

En Producto existía: id_producto → id_categoria → categoria_nombre

Solución: Crear tabla Categoría y referenciarla mediante id_categoria como FK

En Compra existía: id_compra → id_proveedor → proveedor_nombre

Solución: Crear tabla Proveedor y referenciarla mediante id_proveedor como FK

En Venta existía: id_venta → id_cliente → cliente_nombre

Solución: Crear tabla Cliente y referenciarla mediante id_cliente como FK

En Venta existía: id_venta → id_empleado → empleado_nombre

Solución: Crear tabla Empleado y referenciarla mediante id_empleado como FK

Para la autenticación al sistema se identificó que el acceso (username, password_hash, rol) no depende directamente de la PK del empleado, por lo que se separó:

Solución: Crear tabla Usuario relacionada 1:1 opcional con Empleado

Caso especial — atributo subtotal en Detalle_Venta y Detalle_Compra:

Existe la dependencia cantidad, precio_unitario → subtotal (derivada matemáticamente)

Justificación para mantenerlo: se conserva como snapshot histórico del cálculo al momento de la transacción y se asegura su consistencia mediante un CHECK (subtotal = cantidad * precio_unitario) a nivel de esquema

Se cumple 3FN